Proxmox Storage DRS -- Internals

How the tool works, for whoever changes it

Bernd Zeimetz bernd@bzed.de

2026-09-05

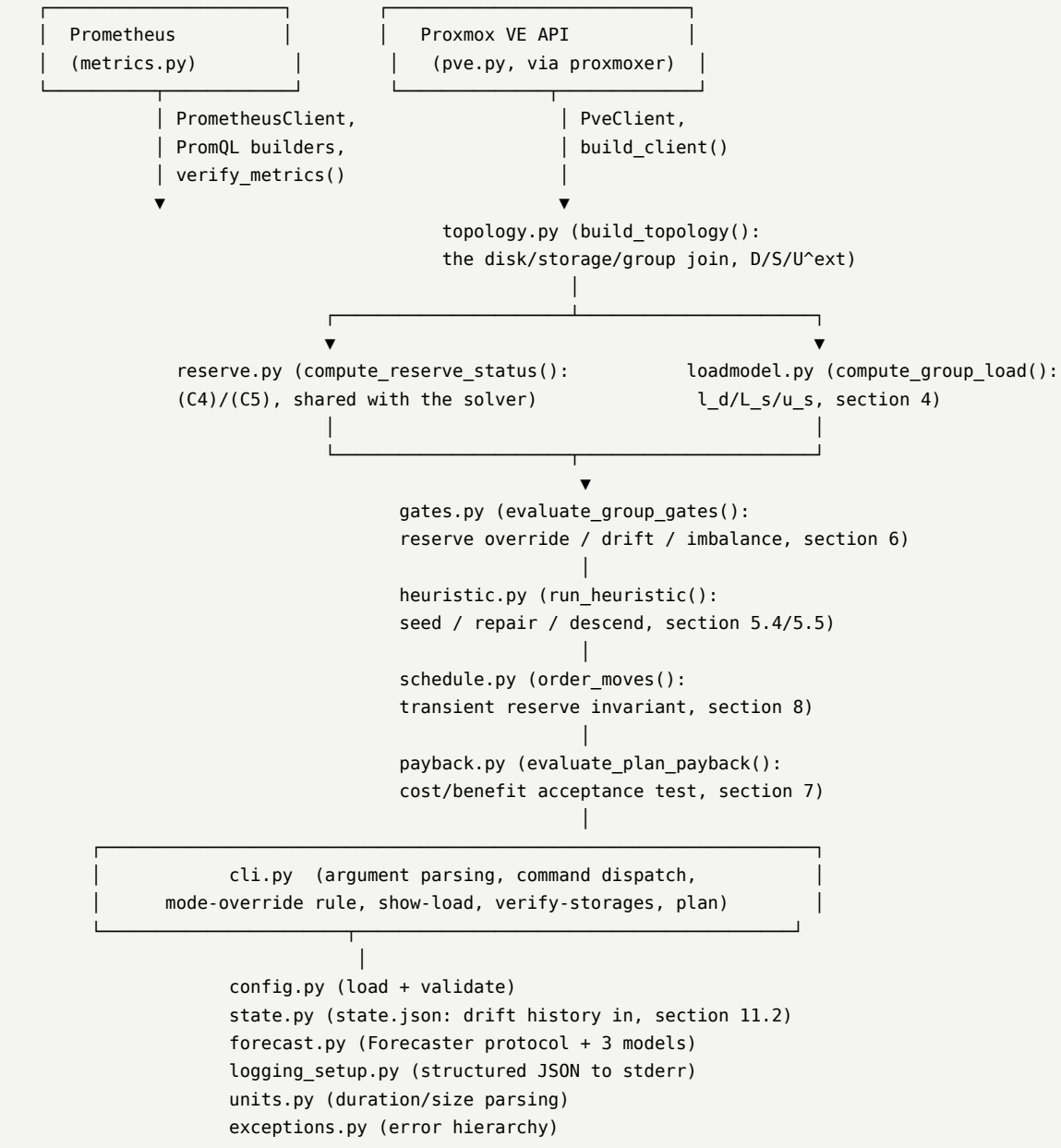
Rendered from `docs/internals/*.md`, SHA-256 `d74b57862a7c191d47514e78c48e197f52cdce9d5802d0de535ee0fec530a9df`
(`sha256sum --check docs/internals.pdf.sha256` in the repository must succeed).

Contents

What does this page answer? Where does an invocation of `pve-storage-drs` go, which module owns which piece, and how much of `IMPLEMENTATION_PLAN.md`'s section 2 architecture actually exists right now?

The pipeline, as specified and as built

`IMPLEMENTATION_PLAN.md` section 2 describes seven stages: collect, join, gate, solve, cost, order, execute. As of this page, stages 1 (collect) through 6 (cost/order — sections 5-8 minus the MILP backend) exist for the heuristic path — the MILP path (an alternative stage-4 backend) and execute are `IMPLEMENTATION_PLAN.md` section 12 phases 6-9 and are not yet written. Do not take this page as a claim that the whole pipeline runs end to end — `../manual/30-safety-and-status.md` is the authoritative per-command status: `plan` is real and prints an ordered, transient-feasible, payback-checked dry-run plan (see `../manual/27-plan.md`), but nothing yet *executes* one (no `apply`). `state.json` itself exists (`state.py`) and is read by both `show-load` and `plan` for real drift history — see `15-state.md` and `80-gates.md` — but nothing writes it yet, since only `execute.py` (not yet written) has a reason to.



Module layout, as it stands

Module	Responsibility	Plan section
`exceptions.py`	The `DrsError` hierarchy every deliberate failure raises	AGENTS.md section 5
`units.py`	Duration/size parsing (`"24h"`, `"200MiB"`) and human-readable formatting	section 11
`config.py`	Load `/etc/pve/drs.yaml`, jsonschema + semantic validation, the frozen dataclass config model	section 11
`config_schema.json`	The jsonschema structural half of validation	section 11.1
`state.py`	`state.json`: the load vector as of the last executed balance, cooldowns, the node-local advisory `flock()` — reading degrades, writing raises	section 11.2
`forecast.py`	The `Forecaster` protocol, `required_range_seconds`, and `quantile`/`seasonal_naive`/`holt_winters`	section 10
`logging_setup.py`	Structured JSON logging to **stderr**	section 2.1 (amended, see `40-cli-and-logging.md`)
`metrics.py`	`PrometheusClient`, PromQL construction, `verify_metrics()`, `compute_disk_coverage()`	sections 3.1-3.4
`pve.py`	`PveClient` (built on `proxmoxer`), `build_client()`	section 3.5
`topology.py`	`build_topology()`: the disk/storage/group join, `D`, `S`, U^{sext}, (C2) pins	sections 3.5-3.7, 5.1, 5.3 (C2)
`reserve.py`	`compute_reserve_status()`: (C4)/(C5), shared by `show-load` today and the solver later	section 5.3 (C4)/(C5)
`loadmodel.py`	`compute_group_load()`: the raw-series-to-ℓ_d blend, `min_coverage` rejection, current `L_s`/`u_s`	section 4
`gates.py`	`evaluate_group_gates()`: reserve override, drift, imbalance — the act/no-act verdict, with reasoning	section 6
`heuristic.py`	`run_heuristic()`: seed/repair/descend, and `evaluate_assignment()`, the section 5.4 objective shared with the (unwritten) MILP path	sections 5.4/5.5
`schedule.py`	`order_moves()`: transient-feasible ordering of a target assignment's moves, deadlock reporting	section 8
`payback.py`	`evaluate_plan_payback()`: the cost/benefit acceptance test, with a reserve-override exemption mirroring `gates.py`'s	section 7
`cli.py`	Argument parsing, command dispatch, `--manual`, the mode-override rule, `show-load`, `verify-storages`, `plan`	section 11.3

Not yet written: `optimize.py`, `execute.py` — and, within modules that do exist, cooldown handling in `gates.py`/`topology.py`/`heuristic.py` (needs `state.json`'s cooldown timestamps to be *read* somewhere, which nothing does yet even though `state.py` stores and round-trips them fine; see `15-state.md` and `80-gates.md`), heuristic step 4 "polish" and (C2) format-compatibility eligibility in `heuristic.py` (see `90-heuristic.md`), concurrent scheduling, priority-2 ordering and staging in `schedule.py` (see `95-`

schedule.md), and the payback re-solve-and-retry loop plus the section 7.3 saturation check in payback.py (see 96-payback.md).

Why config.py depends on forecast.py

config.py's semantic validation (_check_forecast_window) must reject a window.lookback too short for the configured forecast.model at startup (section 11.1) — that check needs forecast.required_range_seconds(). Since forecast.py in turn needs config.ForecastConfig for its type signatures, the import is deferred to inside the validation function rather than at module level, breaking what would otherwise be an import cycle. See the comment at the top of config.py's _check_forecast_window if you touch this.

Where to read next

- 10-configuration.md — how a YAML file becomes a validated Config, and where every default actually lives.
- 15-state.md — state.json's dataclasses, why reading it degrades but writing raises, the real flock() lock and the rename-vs -in-place bug it takes to get that wrong, and what show-load/plan actually get from it today.
- 20-forecasting.md — the forecaster protocol and its three implementations.
- 30-metrics.md — the Prometheus client and the six verify-metrics checks.
- 40-cli-and-logging.md — command dispatch, the --mode escalation rule, and why logs go to stderr.
- 50-pve-api.md — the PVE API client, why it is built on proxmoxer, and two things verified against a real cluster.
- 60-topology.md — the disk/storage/group join, the section 5.1.1 foreign-volume accounting, and the shared (C4)/(C5) evaluator.
- 70-loadmodel.md — the section 4 raw-series-to-ℓ_d blend, min_coverage rejection, and why tpmstate0/unusedN are exempt from it but efidisk0 is not.
- 80-gates.md — the section 6 act/no-act verdict, how state.json's drift history now reaches it, and why cooldowns are not in this module.
- 90-heuristic.md — the section 5.4/5.5 objective and the seed/repair/descend search, cross-checked against section 14's exact objective totals, and what "polish" and format eligibility still owe.
- 95-schedule.md — ordering a target assignment's moves under the section 8 transient invariant, why cost_m = z_d is exact today (not an approximation), and why the heuristic accepting a residual violation can still mean the scheduler reports a deadlock.
- 96-payback.md — the section 7 cost/benefit test, the headroom_src/headroom_dst gap in the plan's own cost formula, and why a reserve-fixing plan always passes the economic test.

How a YAML file becomes a validated Config

What does this page answer? What exactly happens between pve-storage-drs -c foo.yaml plan and a type-checked Config object, and where does each of the section 11.1 validation rules actually live? Describes proxmox_storage_drs/config.py and config_schema.json.

Resolution order

resolve_config_path() implements IMPLEMENTATION_PLAN.md section 11's three-source order: --config (cli_path), then $PVE_STORAGE_DRS_CONFIG, then config.DEFAULT_CONFIG_PATH (/etc/pve/drs.yaml). It also returns was_explicit, which load_config() uses to decide the error message on a read failure: an explicitly named path that cannot be read is always fatal with no fallback, per section 11, but the *message* additionally points at config/drs.example.yaml only when the failing path was the unnamed default — there is nothing useful to suggest when the operator named the path themselves.

The two-stage validation

1. **_validate_schema()** loads config_schema.json via importlib.resources (it ships as package data — see pyproject.toml's [tool.setuptools.package-data]) and runs it through jsonschema. Every object in the schema sets additionalProperties: false: a typo'd key is caught here as a validation error, not silently ignored, which is what keeps section 15.1's knob-to-formula table trustworthy (a key with no formula would otherwise go unnoticed).
2. **_build_config()** walks the now-schema-valid raw dict into the frozen dataclasses defined earlier in the module, applying every default inline via .get(key, default). These defaults are also what config/drs.example.yaml documents and what the manual's ../manual/10-configuration.md describes — there is deliberately no second copy of a default anywhere else in the codebase.
3. **_validate_semantics()** runs the section 11.1 rules jsonschema cannot express — cross-field comparisons, or rules needing a value derived from two independently-optional settings. Each rule is a small _check_* function appending to a shared errors/warnings list (kept separate functions rather than one large one, partly for flake8's max-complexity and partly because each is independently testable). errors become one ConfigError naming every problem found, not just the first; warnings are returned to the caller (ResolvedConfig.warnings) for cli.py to log — section 11.1's saturation_load check is the one warning-only rule today.

Every duration or size field is parsed **once**, at this point, via units.parse_duration_seconds/parse_size_bytes, and stored on the dataclass already in its canonical unit (seconds, bytes) — nothing downstream re-parses a string.

Secrets from the environment

_build_config() fills proxmox.auth.password/token_secret from PVE_PASSWORD/PVE_TOKEN_SECRET **only when the file's own value is null** — a secret actually written in the file is used as-is, never silently overridden by an environment variable a deploy script forgot to unset. This one-directional fallback is what section 11's "keep secrets out of /etc/pve" guidance is for: the file can omit the field entirely and rely on the environment, or set it explicitly and take responsibility for the file's own permissions.

What is deliberately *not* checked here

Two section 11.1 rules need live cluster data this module does not have and are therefore **not** part of config.py:

- storage ids actually existing in the cluster;
- execution.source_release.timeout / gates.cooldown_per_storage against a storage's real saferemove_throughput.

Both belong to pve.py/topology.py (which do the live cluster fetch) and pve-storage-drs verify-storages — see ../manual/30-safety-and-status.md for current status.

ResolvedConfig: what the caller actually gets

load_config() returns a ResolvedConfig, not a bare Config: the resolved path and the file's sha256 travel with it, because IMPLEMENTATION_PLAN.md section 11 requires every run to log both — a plan must always be traceable to the exact file that produced it, even though the file is never re-read after startup.

Persistent state: state.json

What does this page answer? What does state.py actually store, how does reading it degrade instead of failing, why is the advisory lock a real flock() rather than a JSON field, and what does today's wiring into show-load/plan actually change? Describes proxmox_storage_drs/state.py.

One dataclass tree, matching section 11.2's JSON exactly

State mirrors the plan's own example document field for field: lock (LockInfo | None), last_balance (at plus a "<group>:<vmid>:<device>" -> ℓ_d load_vector), cooldowns (disk/storage, each a key -> timestamp mapping), inflight_upids, staged_disks. disk_state_key()/ storage_state_key() build the two key shapes section 11.2 specifies — group-prefixed specifically so a vmid reused after a VM is destroyed and recreated in a *different* group cannot collide with its old entry. empty_state() is what a fresh install, or a lost file, degrades to: no lock, no balance, no cooldowns, nothing in flight.

Reading degrades; writing does not

load_state() never raises. A missing file is the ordinary first-run case (silent); a present-but-corrupt, empty, or wrong-schema_version one is still just empty_state(), but logged at WARNING — section 11.2's own words are "losing this file is safe but not free: cooldowns and drift history reset", not "state.json existing and being readable is required to run". This mirrors show-load's choice to degrade a group's load to "unavailable" on a Prometheus outage rather than fail the whole command — neither Prometheus nor state.json is an *explicitly requested* input the way -c/--config PATH is, and only the latter is a hard failure on error (.agents/domain-invariants.md section 9).

save_state_atomic(), by contrast, raises StateError on any failure — writing (or locking) is something a caller actively asked this module to do, not a read with a documented, safe fallback. It writes to a temp file in the same directory, fsyncs, then os.replace()s over the real path, so a concurrent load_state() can never observe a half-written file — which is exactly what lets every read in this codebase skip locking entirely (next section).

Locking: flock() is the mechanism, the JSON lock field is a label

acquire_lock() takes a real, non-blocking fcntl.flock(LOCK_EX | LOCK_NB) on the state file itself. That is the actual, kernel-enforced mutual exclusion section 11.2 asks for, and it is correct by construction for two instances on the *same host* — .agents/domain-invariants.md section 9 is explicit that "the fcntl lock cannot see another node"; cross-node coordination is the separate in-flight-UPID scan (section 13), not this module's job.

The lock: {pid, host, acquired_at} field recorded *inside* the JSON is **descriptive metadata for a human reading the file, not the exclusion mechanism** — by the time this module's code overwrites it, the OS has already granted exclusive ownership, so whatever a previous run last wrote there is necessarily stale and is unconditionally replaced, never inspected to decide whether to proceed. This is also how section 11.2's "if lock.pid is not alive on lock.host, reclaim it" is satisfied, without a separate liveness check to get subtly wrong: a process that died released its flock() the moment its file descriptor closed (any exit, including a crash), so the *next* acquire_lock() call simply succeeds. acquire_lock() returns None — not an exception — when another live instance already holds it, matching section 11.2's "a live PID means another instance is running: exit 0 quietly"; deciding what "quietly" means, and which exit code, is left to the caller.

A real bug this design surfaced during testing, worth remembering if you touch this code: the first implementation had acquire_lock() write its lock metadata via save_state_atomic() — the same temp-file-plus-rename used everywhere else. That silently breaks the lock: a rename replaces the directory entry with a *new* inode that was never flock()'d, so a second acquire_lock() opens that new inode fresh and locks it with no conflict at all — test_a_second_acquire_while_the_first_is_held_returns_none caught this immediately. The fix, _write_state_to_locked_fd(), writes **in place** into the already-open, already-locked file descriptor (lseek to 0, write, ftruncate to the new length, fsync) — never a rename — for as long as the lock is held.

What today's wiring into `cli.py` actually changes

`show-load` and `plan` both call `_last_loads_by_group()` once, right after reading configuration, and use the result for both `loadmodel.compute_group_load()`'s `last_known_loads` and `gates.evaluate_group_gates()`'s `last_load` — one state read per run, one function computing the per-group slice, not two independent implementations (AGENTS.md section 5). Neither command takes the advisory lock: both are read-only reports that never execute a migration, and taking the exclusive lock for one would make an in-progress `apply` block `plan`/`show-load` for no safety reason the plan text supports.

The practical effect: a `state.json` with a real `last_balance` now makes the drift gate genuinely suppress an `ACT` verdict the imbalance alone would otherwise trigger — see `test_show_load_gate_reflects_real_drift_history_from_state_json` and `test_plan_gate_also_reflects_real_drift_history_from_state_json` in `tests/unit/test_cli.py` for the exact before/after. Without a `state.json` on disk (the common case until `execute.py` exists — see below), behaviour is unchanged from before this module existed: `last_load=None`, drift gate skipped, "reserve override, else imbalance".

Deliberately not implemented in this pass

- **Nothing calls `acquire_lock()` or `with_recorded_balance()` yet.** Both exist and are fully tested, but only `execute.py` (phase 7, not yet written) has a reason to take the lock or record a balance — section 11.2 itself says `last_balance` is "updated only after a run that executed at least one migration", and neither `show-load` nor `plan` ever does.
- **Cooldowns are stored and round-tripped (`Cooldowns`, `with_recorded_cooldown()`) but not yet *read* by anything that pins a disk or excludes a migration target.** Section 5.3 (C2)'s "within its per-disk cooldown -> pin to current" belongs with `topology.py`'s other (C2) pin reasons (locked, excluded, snapshotted — see `topology._pin_reason()`); the storage-side "accepts no new incoming moves" belongs with `heuristic.py`'s target eligibility. Both need a `State`/cooldown lookup threaded into a function that does not accept one today — a real, separately-scoped change to two other modules, not this one.
- **`inflight_upids`/`staged_disks`** exist in the dataclass and round-trip correctly, but nothing writes or reads them yet — they belong to `execute.py` (crash recovery, section 13) and `schedule.py`'s staging (section 8.3 option 1, also not implemented), respectively.

Forecasting: the protocol and its three implementations

What does this page answer? How does `forecast.py` decide how much history a model needs, and what does each of `quantile`/`seasonal_naive`/ `holt_winters` actually compute? Describes `proxmox_storage_drs/forecast.py`.

The one rule, in one place

`IMPLEMENTATION_PLAN.md` section 10.1's table says how much history each model needs, independent of `window.lookback` (the *decision* window). That rule has exactly one implementation: three small pure functions, `_quantile_required_range_seconds`, `_seasonal_naive_required_range_seconds` and `_holt_winters_required_range_seconds`, each called from **two** places — the free function `required_range_seconds()` that `config.py`'s startup validation uses (before any `Forecaster` is constructed), and the matching `Forecaster.required_range()` method on the concrete class. Duplicating the arithmetic between those two call sites, rather than sharing the pure function, is exactly the kind of "second copy of a rule" AGENTS.md section 5 forbids — a change to the Holt-Winters requirement made in only one of them would silently desynchronize config validation from what the forecaster itself believes it needs.

Why the upper bound, never the point estimate

Every `Forecaster.predict()` returns a `Forecast(point_estimate, upper_bound)`. Nothing in this module enforces that callers use `upper_bound` — that discipline belongs to the caller (the optimizer and the

section 7.3 saturation guard, neither written yet). The asymmetry is stated in the module docstring and repeated here because it is easy to get backwards under time pressure: overestimating costs a slightly worse balance, underestimating risks scheduling a mirror onto a storage that is about to saturate.

`QuantileForecaster`

The default. `predict()` takes the `window.quantile`/`window.upper_quantile` percentiles of the raw series with no fitting at all. `_quantile()` implements linear-interpolation percentile matching `numpy.percentile`'s default method, by hand — deliberately not using numpy, since the tool must stay usable with only `requests` + `ruamel.yaml` + `jsonschema` installed (`IMPLEMENTATION_PLAN.md` section 2.1). `horizon` is accepted (to satisfy the `Forecaster` protocol) and immediately discarded: the quantile model has no notion of a horizon-dependent forecast.

`SeasonalNaiveForecaster`

Groups historical samples by hour-of-day (`int((ts // 3600) % 24)`) and takes the median as the point estimate, the configured upper quantile across the same bucket as the bound. `now_epoch_seconds` selects which hour-of-day bucket the *current* moment falls into; `predict()` ignores everything outside that bucket. A bucket with no samples returns `Forecast(0.0, 0.0)` rather than raising, matching `IMPLEMENTATION_PLAN.md` section 4's rule for an idle group's raw quantities: no data is zero, not an error, at this layer (the caller decides whether zero is trustworthy).

`HoltWintersForecaster`

`statsmodels` is an optional dependency (`Suggests`, not `Depends` — see `debian/control`), so it is imported **only inside `predict()`**, never at module level; `debian/tests/import-all` is what would catch a regression here. Two situations fall back to `QuantileForecaster`, both logged at warning level rather than failing silently:

1. fewer than `2 * seasonal_periods` samples in the series (section 10.2's own rule — a fit on too little data is worse than no fit);
2. `statsmodels` is not importable at all.

The upper bound is `point_estimate + residual_z * stdev(in-sample residuals)` — not derived from the model's own confidence interval, which `statsmodels`'s `ExponentialSmoothing.fit()` does not expose directly for every configuration. `residual_z` (`forecast.holt_winters.residual_z`, default `2.0`) is what an operator tunes if this bound turns out too tight or too loose in practice; there is no plan-mandated backtesting gate on it yet (`IMPLEMENTATION_PLAN.md` section 10.2's backtest-validation requirement is not implemented — tracked for the dedicated forecasting phase, section 12 phase 9).

`build_forecaster()`

The one factory function that turns a `ForecastConfig` plus the ambient `window`/`metrics` values into a live `Forecaster` instance. Nothing else in the codebase should construct a `QuantileForecaster`/`SeasonalNaiveForecaster`/ `HoltWintersForecaster` directly once a caller exists that needs one from config — route it through here so the model-name-to-class mapping stays in one place.

The Prometheus client and `verify-metrics`

What does this page answer? How does `metrics.py` talk to Prometheus without ever touching the network in a test, and what does each of the six `verify-metrics` checks actually query? Describes `proxmox_storage_drs/metrics.py`.

_SessionLike / _ResponseLike: why not requests.Session directly

PrometheusClient.__init__ accepts session: _SessionLike | None, a Protocol defined in this module with only the one get(...) method the client actually calls — not requests.Session itself. .agents/testing.md forbids any test talking to a real Prometheus; a structural protocol lets a test hand in a small dataclass with a matching get() instead of mocking or subclassing the real (network-capable) session. PrometheusClient() with no session argument still constructs a real requests.Session(), which satisfies the protocol structurally — nothing special is needed to make the real thing fit.

_get(): one request path, three endpoint shapes

Every one of instant_query, range_query and label_values goes through the private _get(), which does the HTTP call, checks the status code, parses JSON, checks Prometheus's own status: "success" field, and returns payload["data"]. That field's *shape* genuinely differs by endpoint — a dict with a result list for query/query_range, a bare list for label/<name>/values — which is why _get() is typed Any rather than picking one shape; each public method knows which shape it asked for and narrows accordingly. Every failure mode (a transport error, a non-200 status, unparseable JSON, an unsuccessful Prometheus response) raises MetricsError from exactly this one place, so a caller catching MetricsError around any of the three public methods sees every failure mode uniformly.

PromQL construction

build_rate_promql() and build_quantile_over_time_promql() are pure string-building functions, deliberately factored out of any query-issuing code so they are unit-testable without a fake session at all — see IMPLEMENTATION_PLAN.md section 3.4 for the exact expressions they implement. raw_metric_name() is the one place that maps a RAW_METRIC_FIELDS entry ("read_ops", ...) to the configured metric name on a MetricsConfig, via getattr — this is what lets verify_metrics() iterate "every configured raw metric" without hand-listing the six field names a second time anywhere in this module.

verify_metrics(): six checks, six functions

Each _check_* function implements exactly one IMPLEMENTATION_PLAN.md section 3.3 numbered check and returns Findings (plus, where relevant, the data the report carries forward):

Function	Section 3.3 step	Returns
_check_metric_names_exist	1	findings only
_check_sample_series	2 + 3 (labels present)	findings, {metric_name: sample_labels}
_check_device_label_collision	4	one Finding or None (pure, no I/O)
_check_coverage	5	findings, {DiskKey: coverage_fraction}
_check_observed_spacing	6	findings, observed_spacing_seconds

_check_coverage and _check_observed_spacing both use metrics.read_ops as the one representative metric rather than probing all six: a coverage or spacing gap is a property of the underlying Telegraf scrape, not of which of the six raw quantities is read, so probing all six would be six times the Prometheus load for no additional information.

VerifyMetricsReport.ok is True iff no Finding has level == "error" — cli.py's verify-metrics handler uses exactly this property to decide the process exit code, so there is one definition of "the verification passed."

What cli.py does with the report

_handle_verify_metrics in cli.py constructs a PrometheusClient from resolved.config.prometheus, calls verify_metrics(), and renders either the human form ([level] message lines) or, under --json, a JSON-serializable dict — DiskKey keys are turned into "vmid:device" strings there, since JSON object keys must be strings and this module keeps no opinion about output formatting.

Command dispatch, the mode-override rule, and why logs go to stderr

What does this page answer? How does cli.py turn argv into a dispatched command, what exactly does overriding --mode log, and why does structured logging go to stderr instead of the stdout the plan originally specified? Describes proxmox_storage_drs/cli.py and proxmox_storage_drs/logging_setup.py.

One parser, global options first

build_parser() defines every global option (-c/--config, --group, --mode, --json, -v/--quiet, --version, --manual) on the **top-level** ArgumentParser, not on the subparsers, which is what makes argparse itself enforce IMPLEMENTATION_PLAN.md section 11.3's "global options before the subcommand" for free — there is no hand-written ordering check anywhere. --help's defaults come from the real default= values passed to add_argument, combined with argparse.ArgumentDefaultsHelpFormatter; there is deliberately no second, hand-maintained list of options and defaults anywhere in this module, the manpage, or the manual (AGENTS.md section 8.5).

main() handles --version and --manual/help **before** requiring or loading any configuration — printing the version or the manual page must never depend on /etc/pve/drs.yaml existing. Only once a real subcommand is present does it call config.load_config().

Command handlers: a dict, not a chain of ifs

_COMMAND_HANDLERS: dict[str, CommandHandler] maps each subcommand name to a function of (ResolvedConfig, argparse.Namespace, effective_mode) -> int. Every subcommand not yet implemented (apply, explain — see ../manual/30-safety-and-status.md for which ones that currently is) gets a handler from _make_not_yet_implemented_handler(name), a closure factory rather than a lambda with a default-argument trick, because mypy's strict mode cannot infer a bare lambda's parameter types cleanly here — see the comment at that function if you are tempted to simplify it back to a one-liner. _COMMAND_HANDLERS["verify-metrics"] is overwritten with the real _handle_verify_metrics once metrics.py exists to back it. Adding a new implemented command is exactly this: write the handler, assign it into the dict, done — main()'s dispatch does not change.

--group: filtered once, right after build_topology()

_filter_groups(topology, args.group) is the one place --group is actually read. show-load, verify-storages and plan each call it immediately after build_topology(), replacing that Topology's groups tuple with the (still topology-order) subset named — every render function downstream just iterates topology.groups as before and needs no --group awareness of its own. verify-metrics does not call it: that command validates configured metric/label names against Prometheus directly and never iterates groups at all, so there is nothing for --group to restrict there. A name that matches no configured group raises DrsError rather than silently producing an empty report — the same "an explicitly named thing that doesn't resolve is a hard failure" rule config.load_config() already applies to -c/--config PATH (REVIEW.md R-03: --group was previously parsed and documented, but no handler ever read args.group at all).

The `--mode` escalation rule

`apply_mode_override(configured_mode, override)` is the entire implementation of section 11.3's rule: moving *down* the ordering `dry-run < confirm < auto` (toward more caution) logs at `INFO`; moving *up* it (toward less caution — removing a barrier the operator's own config set) logs at `WARNING`, naming both values. This is deliberately a small pure-ish function (its only side effect is the log call) rather than inlined into `main()`, so it is unit-testable against a `caplog` fixture without constructing a full CLI invocation.

Why logs go to stderr, not stdout

`IMPLEMENTATION_PLAN.md` section 2.1 originally specified stdout for structured JSON logs. This was amended in the same commit that added `logging_setup.py`: `--json` (section 9.5) emits the plan report on **stdout**, and interleaving log lines with that report on the same stream would make it unparseable by anything downstream. Logging instead goes to **stderr**; a systemd service unit captures both streams into the same journal regardless, so nothing about "captured by journald" is lost. This is the one place in the codebase where the implementation and `IMPLEMENTATION_PLAN.md` disagree with the *original* plan text on purpose — AGENTS.md section 7 rule 2 is why the plan text itself was corrected in the same commit rather than left to silently diverge from the code.

`configure_logging()` is called exactly once, from `main()`, after the `--version`/`--manual` early exits (which must not touch logging configuration at all) and before any config-dependent code runs, so that a config-loading failure is itself logged consistently with everything after it.

`JsonFormatter`: what ends up in one log line

Every field on a `logging.LogRecord` that is *not* one of the standard attributes (computed once, at import time, into `_STANDARD_RECORD_ATTRS`) is folded into the JSON payload — this is what lets any module call `logger.info("...", extra={"event": "gate_decision", "threshold": 0.2})` and have `event`/`threshold` appear as top-level JSON keys with no formatter changes required. `sort_keys=True` keeps output byte-stable for anything that diffs log lines in a test or a log-aggregation pipeline.

`--manual`: preferring `man(1)`, falling back only when necessary

`show_manual()` tries `man pve-storage-drs` first, via `subprocess.run` rather than `os.execvp`: `man(1)` itself detects a non-tty stdout and disables its pager automatically, which is what satisfies "never answer with a URL alone" (AGENTS.md section 8.5) whether the invocation is interactive or piped — there is no need for this module to duplicate that tty detection. `_fallback_manual_text()` is reached only when `man(1)` itself is missing or has no entry (an uninstalled source checkout, or a minimal system with no man-db): it walks up from `cli.py`'s own file location looking for `man/pve-storage-drs.1.md` in a source tree, and only if that also fails prints a short message naming the real installed-package path and the in-tree file — a local, actionable path, never a bare URL.

The Proxmox VE API client

What does this page answer? Why is `pve.py` built on `proxmoxer` instead of a hand-rolled ticket/CSRF client, and what does each of its methods actually call? Describes `proxmox_storage_drs/pve.py`.

Why `proxmoxer`

`IMPLEMENTATION_PLAN.md` section 3.5 explains the decision in full; the short version is `proxmoxer`'s **backend abstraction**. The same `ProxmoxAPI` attribute-chaining interface (`api.nodes(node).qemu(vmid).config.get()`) is available over plain HTTPS (what `build_client()` uses today) or over SSH, either shelling out to the system's own `ssh+pvesh` (`openssh`) or with an in-process client (`ssh_paramiko`). A deployment that cannot open the API port to the management host becomes a different keyword argument in `build_client()`, with no change anywhere else — not to `PveClient`'s methods, and not to any of their callers.

`build_client()`: the one place auth is assembled

`config.py`'s `ProxmoxConfig` keeps `verify_ssl` (bool) and `ca_file` (path-or-`None`) as two separate knobs, because that is how an operator thinks about them; `requests` (and so `proxmoxer`'s https backend) wants one value, a bool or a CA-bundle path, passed as `verify`. `build_client()` is the one place that reconciles the two. It is also the one place `proxmox.auth.token_id`'s `user@realm!tokenname` format is split into `proxmoxer`'s separate `user`/`token_name` keyword arguments — never duplicate that parsing anywhere else.

Every failure `ProxmoxAPI(...)` itself can raise (`proxmoxer.AuthenticationError`, or a `requests.RequestException` if the host is simply unreachable) is caught here and re-raised as `PveApiError`, so every caller of `build_client()` and every `PveClient` method has exactly one exception type to catch.

`PveClient`: one method per section 3.5 endpoint

Every method is a single API call, wrapped through the private `_call()` helper, which is the one place `proxmoxer.ResourceException` (HTTP-level API errors) and `requests.RequestException` (transport failures `proxmoxer`'s https backend does not wrap itself) both become `PveApiError`. No method caches anything, and the per-run topology cache belongs to `topology.py` (section 3.5), which is also where the bounded thread pool for the O(VMs) config fetch lives (`60-topology.md`, REVIEW.md P-02).

`_call()` does retry exactly one failure mode: `proxmoxer.AuthenticationError` gets one reauthenticate-and-retry cycle (REVIEW.md P-01) before becoming a `PveApiError`, via a `reauthenticate` callback `build_client()` wires in — a full fresh login through `_build_api()` again, not a reuse of the rejected ticket. This is not a general retry policy (a `ResourceException` or a transport failure still fails immediately, on the first attempt); it exists specifically because a ticket can expire for reasons with nothing to do with the call that hits it — a long `apply --confirm` wait, a suspended process, a clock jump — and `proxmoxer`'s own lazy per-request renewal only notices the age of its *own* clock, not whether the server-side ticket actually outlived a gap that long. A test double built with a bare `PveClient(fake_api)` (no `reauthenticate=`) gets none of this — it behaves exactly as it did before P-01, which is what every existing fake in `tests/unit/fakes.py` relies on.

`build_client()` also applies `proxmox.ticket_refresh_seconds` to the constructed session, best-effort: `proxmoxer` 2.x has no constructor argument for a password/ticket auth's refresh interval (it is a hard-coded `renew_age = 3600` class attribute on `ProxmoxHTTPAuth`, checked lazily on whichever request happens to run after it elapses), so `_apply_ticket_refresh_seconds()` reaches into `api._backend.auth` — undocumented `proxmoxer` internals, not its public interface — to override that instance's `renew_age` after login. Deliberately narrow: it only ever *tightens* proxmoxer's own schedule, never replaces its refresh logic, and if a future `proxmoxer` version reshapes this (or the backend is not `https`, or the auth is API-token, which has no ticket at all), the `getattr`/`hasattr` guards make it a silent no-op — the P-01 reauthenticate-and-retry above still catches the case this can't reach.

`storage_definitions()` deliberately uses `GET /storage` (the list form), never the singular `GET /storage/{id}`, even though only one storage's config is needed in some callers. This was verified empirically against a real PVE 9.2.11 cluster during development ([[dev-cluster-access]] in the maintainer's notes, not reproduced here): an API token granted only `Datastore.Audit` can list every storage's full config — including `saferemove`/`saferemove_throughput` — via `GET /storage`, but the identical data via `GET /storage/{id}` for the same storage returns `403 Forbidden (Datastore.Allocate)`. The single-item route apparently backs an edit-UI flow gated by the ability to change the config, not merely read it. Since the list form returns the same per-storage object for every storage in one call, there is no reason to ever call the singular form, and using it would quietly force a Storage DRS token to hold more privilege than the tool actually needs. `IMPLEMENTATION_PLAN.md` section 3.5 was corrected to match once this was

found — see that section's note.

`storage_content()` returned an empty list — not an error — on every storage, node and content-type filter tried, until the token also held `Datastore.Allocate` on that storage. This was genuinely surprising and worth calling out precisely because it is a *silent* false negative: `Datastore.Audit` alone gives a clean `200` with `{"data": []}`, which looks exactly like "this storage legitimately has no tracked content" rather than "the caller cannot see it." It was not guessed at — see `IMPLEMENTATION_PLAN.md` section 3.5's note, which records the exact before/after (0 entries with Audit only, 37 and 1 entries respectively for two real storages once `Datastore.Allocate` was added) on the same live cluster. **The practical consequence: `pve.py`'s own "keep the token to Audit-only" goal for `storage_definitions()` does not extend to `storage_content()`** — a Storage DRS token genuinely needs `Datastore.Allocate` (bundled with `Datastore.Audit` in one role) on every storage it manages, or `topology.py` will silently compute U^{sext} (section 5.1.1) as if every storage were empty of untracked volumes, with no error to notice by. `docs/manual/00-installation.md` states the exact minimum ACL grant an operator needs; this is not a place to economize on privilege out of a preference that turned out not to match reality.

Two more things about Proxmox's ACL model this surfaced, both now reflected in the manual's token-setup instructions: **an API token's effective permission is the *intersection* of its owning user's permissions and whatever is granted to the token itself** when privilege separation is on (the default) — granting a role to only one of the two has no effect, both need it; and the grant that was confirmed to work was made explicitly at each `/storage/{id}` rather than only at the parent `/storage` path, so that is what the manual instructs, rather than relying on propagation that was not cleanly isolated as working or not.

`move_disk()`: the one and only bytes/s -> KiB/s conversion

Section 9.2 is explicit that `bwlimit`'s bytes/s-to-KiB/s conversion happens "at the call site and nowhere else." `PveClient.move_disk()` is that call site: every caller in this codebase, present and future, passes `bwlimit_bytes_per_sec` in the same unit `migration.bwlimit_bytes_per_sec` already uses, and the division by 1024 (rounded, since the API wants an integer) happens exactly once, inside this method. `format` defaults to `None` and is only ever forwarded when a caller explicitly passes one — `move_disk` without `format=` preserves the source format, which is what section 3.5 requires by default.

Building the disk/storage/group model

What does this page answer? How does `topology.py` turn `pve.py`'s raw API responses into the per-group D/S sets the rest of the engine needs, and where does each `IMPLEMENTATION_PLAN.md` section 5.3 ($C2$) pin condition actually get decided? Describes `proxmox_storage_drs/topology.py` and `proxmox_storage_drs/reserve.py`.

`D` means every disk this module places, pinned or not

This is stated at the top of `topology.py`'s own docstring because it is the single easiest thing to get backwards: D (section 5.1) is *every* disk this module puts into a group's `Group.disks`, whether or not ($C2$) pins its placement. A disk this module never sees at all — a stopped VM excluded by `exclude.running_only`, or a disk whose current storage is not in any configured group — is what "foreign" (U^{sext}, section 5.1.1) means. Config-excluded disks (`exclude.vmids`/`exclude.disks`/tags) are *not* foreign: ($C2$) pins them into D specifically so their bytes still count in ($C4$)/($C5$) and their fragmentation toward K (section 3.6, "Pinned disks are modelled, not ignored"). An earlier draft of section 5.1.1 listed config-excluded disks as foreign, which directly contradicted ($C2$) — that self-contradiction was found and fixed in the same commit that first implemented this join; see that section's note if you need the history.

One pass, in the order section 3.5 lists

build_topology() fetches everything it needs exactly once, in five stages (_fetch_cluster_data, then a loop over client.vm_resources() calling _collect_vm_disks per VM, then _build_storages):

1. storage_definitions() (the list form — see 50-pve-api.md), validated against the configured groups (_validate_group_storages): a storage that does not exist, or that lacks images in its content types, is a fatal TopologyError (section 11.1: "storage ids exist in the cluster"). A storage that exists but is not marked shared is a warning, not a fatal error — plausible, if unusual, for a single-node group.
2. storage_resources() once, to pick one active node per storage (_pick_active_node, preferring one reporting status: "available").
3. storage_content() and storage_status() per group storage.
4. vm_config() and vm_snapshots() per considered VM. A VM is *not* fetched at all when exclude.running_only is set and it is stopped — the one case cluster/resources's own fields can decide without a config fetch. Every other VM is fetched, including config-excluded ones: whether a specific disk turns out to be "ungrouped" is only knowable after seeing which storage it is actually on, and a config-excluded VM's disk still needs its size for (C4)/(C5) (previous section).

   This is the one step that runs across a bounded thread pool (config.proxmox.read_workers, REVIEW.md P-02): section 3.5's whole point is that GET .../qemu/{vmid}/config has no batch form, so for a several-hundred-VM cluster this is the read phase concurrency actually helps. It is split into _fetch_vm() (network I/O only, run by the pool, one call per considered VM) and _join_vm_disks() (pure — the same function the old sequential _collect_vm_disks() was renamed from, unchanged in what it computes). ThreadPoolExecutor.map() returns each VM's fetch in the same order client.vm_resources() listed it, even though the fetches themselves complete in whatever order the pool schedules them, so the join phase runs single-threaded and in the original order — disks_by_group, referenced_volids and warnings end up byte-for-byte identical to a fully sequential run regardless of read_workers or of thread scheduling, and never need a lock. PveClient itself may reauthenticate mid-fetch if several threads hit an expired ticket at once (50-pve-api.md's P-01 note); that reauthentication is what its own lock serializes, not this join.
5. Non-QEMU resources (type != "qemu") are skipped outright — this tool never touches LXC containers, and section 3.5's read/write paths are qemu-only throughout.

The lock field comes from vm_config(), not a separate call

Section 9.3's own pseudocode says lock is readable from either /qemu/{vmid}/config or /status/current. _collect_vm_disks reads it from the config response already being fetched for the disk join, so planning-time lock detection costs no extra API call. /status/current is still needed later, but only immediately before a specific move (section 9.2's pre-move re-validation, execute.py, not yet written) — never here.

Pin priority: _pin_reason()

One function, checked in the fixed order section 5.3 (C2) lists its conditions: config exclusion (VM-level, then exclude.disks), a real snapshot or an unreferenced companion volume (section 3.7, _disk_snapshot_or_orphan_reason — the "current" entry GET .../snapshot always returns, verified on a live cluster, is filtered out before counting), a VM lock, then an unusedN disk when exclude.include_unused_disks is false. A disk gets at most one reason; the first that applies wins, matching how an operator would explain it.

Sizes: content is authoritative, config is the fallback

`_resolve_disk_size_and_format` looks up the volume in that storage's content listing first (section 3.5: authoritative for what is actually allocated) and falls back to parsing the VM config's own `size=` only when the volume is missing from content — logged as a warning naming the disk, since assume-thick-provisioning sizing from a stale or unlisted config value is a real, if rare, source of drift. Two independently verified reasons this fallback path is not merely defensive: the `Datastore.Allocate`-vs-`Audit` privilege gap (`50-pve-api.md`), and an `unusedN` volume, deleted directly on the storage backend outside Proxmox (confirmed with the cluster's operator), that PVE's own config still referenced and that PVE itself never noticed or refused to boot the VM over — `IMPLEMENTATION_PLAN.md` section 3.6's note. `_parse_pve_config_size_bytes` parses PVE's own config-file size suffixes (`512G` meaning binary GiB, no explicit `i`) — deliberately not `units.py`'s parser, which is for *this project's* config file, a different and coincidentally similarly-shaped format.

U^{ext}: everything not referenced

Every disk actually placed into a group's `D` records its volid in `referenced_volids[storage_id]` as it is processed. `_build_storages` then sums the size of every content entry on that storage whose volid was never referenced — orphans, templates, other groups' foreign volumes, and disks of VMs this run never fetched (stopped-and-excluded, or ungrouped) all fall out of this one computation with no special-casing per category, which is exactly section 5.1.1's definition read literally.

`reserve.py`: one (C4)/(C5) evaluator, shared

`compute_reserve_status()` is deliberately its own module, not a method on `Storage`: the solver (`optimize.py`/`heuristic.py`, not yet written) will need to evaluate the identical formula against a *candidate* assignment, not only the current one, and `pve-storage-drs show-load`'s reporting needs it against the current assignment today. Both call the same function (AGENTS.md section 5) — `show-load`'s use of it is already exercised end-to-end (see `cli.py`'s `_render_show_load_human/_json`). The section 14 worked example's initial state (`tests/unit/test_reserve.py`) is the proof this reproduces the plan's own arithmetic exactly, not merely a self-consistent unit test.

The load model

What does this page answer? How does `loadmodel.py` turn six raw Prometheus series into the single per-disk ℓ_d section 4 defines, and what happens when the data for one disk cannot be trusted? Describes `proxmox_storage_drs/loadmodel.py`.

One function, one group, already-built input

`compute_group_load()` takes an already-built `topology.Group` (one group's `D`/`S`, section 5.1) and a `metrics.PrometheusClient` and returns a `GroupLoad`: one `DiskLoad` per disk (ℓ_d) and one `StorageLoad` per storage (`L_s`, `u_s`), plus the group's u^* and whether the whole group is idle. It never touches the PVE API and never builds topology itself — matching `reserve.py`'s identical "given the join, evaluate one formula" framing, not a coincidence: both are meant to be called again, unchanged, by the solver once it exists.

Six queries, not six-times-groups queries

The six raw quantities (section 3.4) are fetched with **one instant query each**, unfiltered by group — `sum by (vmid, device) (...)` already returns every disk Prometheus currently reports, and picking out one group's keys in Python is free, whereas six *more* Prometheus queries per group is not. `read_time_ns`/`write_time_ns` are converted from nanoseconds to seconds here, engine-side (`_combined_raw`), for the same reason F-03 moved `read_factor`/`write_factor` engine-side: one tested constant beats the same `/1e9` repeated across deployed PromQL strings.

Known limitation, not a bug: a config with more than one group re-runs these same six unfiltered queries once per group — correct (each group's coverage/rejection decisions are independent) but wasteful for a many-group cluster. Fetching once and slicing per group would be a straightforward follow-up if a real deployment's group count ever makes this Prometheus load worth avoiding; nothing about `compute_group_load()`'s per-group signature forces the current one-fetch-per-call shape, it is just what the first implementation does.

Coverage rejection excludes a disk from the group total, not just from its own ℓ_d

Section 3.4: "reject a disk whose sample coverage over `W` is below `window.min_coverage` and fall back to its last known load from `state.json`... never treat missing data as zero load." Two things worth being explicit about, since the plan states the rule but not this mechanism:

1. **The rejected disk's raw contribution is excluded from `T_g`/`O_g`/`B_g` entirely**, not merely capped or zeroed in place — one noisy or half-missing series must not bias every *other* disk's normalized share. `ℓ_d` for every accepted disk is computed from the accepted-only totals.
2. **The rejected disk's own `ℓ_d` is substituted afterward**, from `last_known_loads` if the caller has one (keyed by `topology.Disk.key`), or flagged `0.0` if not. `DiskLoad.flagged_reason` is set either way, so a caller can never mistake a flagged `0.0` for a genuinely idle disk — `show-load` renders it as an explicit ⚠ line, never silently.

`state.py` (section 11.2) now provides the real `last_known_loads` source: `cli.py`'s `show-load` and `plan` both pass `state.load_vector_for_group(state, group.name)` through unchanged, exactly the mapping this function already expected (see `docs/internals/15-state.md`). `compute_group_load()` itself needed no change at all — the parameter was shaped for this from the start. A group with no recorded balance yet (a fresh `state.json`, or none on disk) still gets `None`, and every coverage-rejected disk with no seed is flagged `0.0`, exactly as before.

tpmstate0/unusedN are never rejected; efidisk0 is

Section 3.4's own note: `tpmstate0` and `unusedN` are not QEMU block devices and legitimately emit no `blockstat` series — `ℓ_d = 0` for them is correct, not a data-quality problem. `_is_metrics_expected_absent()` encodes exactly this device-name distinction so these two kinds are never flagged for low coverage, while `efidisk0` — a real QEMU drive that *should* appear — is held to the same `min_coverage` bar as `scsi`/`virtio`/`ide`/`sata`. Getting this backwards either way is a real bug: exempting `efidisk0` would hide a genuine metrics gap on it; not exempting `tpmstate0`/`unusedN` would spam a flag on every run for every cluster, for a "gap" that isn't one.

T_g = 0 is "idle", computed only from accepted disks

`GroupLoad.idle` is `True` exactly when every *accepted* disk's `raw_t` is zero — the guard section 4 requires ("if `T_g = 0` the entire group is idle, skip it"). An all-rejected group (e.g. a total Prometheus outage mid-window) also comes out `idle=True` by this same computation, which is not quite literally accurate (the truth is "unknown", not "idle") but is harmless: both mean "do not act", which is the only decision that follows from either. A caller that wants to distinguish the two can — every disk's `flagged_reason` is still there to check.

L_s/u_s are the *current* assignment, not a candidate one

`StorageLoad` sums `ℓ_d` over disks whose `Disk.current_storage` already points at that storage — today's state, exactly like `reserve.py`'s `compute_reserve_status()`. The solver (`optimize.py`/`heuristic.py`, not yet written) will evaluate the identical per-disk `ℓ_d` values against a *candidate* assignment instead; nothing in `loadmodel.py` needs to change for that, since `ℓ_d` itself does not depend on which storage a disk is currently on.

What is still missing from phase 3

IMPLEMENTATION_PLAN.md section 12 phase 3's own "done when" is "correct act/no-act decision per group, with the reasoning shown" — that is the section 6 drift/imbalance gates, cooldowns, and the reserve override, none of which exist yet. loadmodel.py is the load computation those gates consume, not the gates themselves; pve-storage-drs show-load reports ℓ_d/L_s/u_s today, but no command yet decides whether a group should be re-balanced.

Gating: deciding whether to act at all

What does this page answer? How does gates.py turn a loadmodel.GroupLoad and per-storage reserve.ReserveStatus into the section 6 act/no-act verdict, and why isn't cooldown handling here? Describes proxmox_storage_drs/gates.py.

One function, three gates, in the order section 6 lists them

evaluate_group_gates() is pure — no network I/O, no state.json access — taking a group's already-computed GroupLoad, a storage id -> ReserveStatus mapping (the caller's job, via reserve.compute_reserve_status() — this module never recomputes it, per AGENTS.md section 5's "one implementation of every rule"), the group's GatesConfig, and the load vector recorded at the group's last *executed* balance (last_load, None if there isn't one). It checks, in section 6's own order, stopping at the first that decides:

1. **Reserve override.** Any storage in reserve_statuses with .violated set forces act=True immediately, bypassing drift and imbalance entirely — section 13's "safety is not subject to hysteresis" made literal, not just documented intent.
2. **Drift gate.** $\|\ell\_now - \ell\_last\|_1 / \|\ell\_last\|_1 \geq$ gates.drift_threshold, vectors aligned over the **union** of disk keys (a disk absent from one side contributes its full load in the other as drift — a new or deleted disk is a genuine change to the group's I/O profile). last_load=None skips this gate outright, per section 6's own degenerate-case table — not "treat as zero drift", which would make an operator's very first run fail to act on an already-imbalanced cluster.
3. **Imbalance gate.** (max_s u_s − min_s u_s) / u* ≥ gates.imbalance_threshold, u* from GroupLoad.average_utilization. u* == 0 (idle group, section 4) always means no-act — there is nothing to spread evenly.

last_load is real now, for show-load and plan

state.py (section 11.2) exists: cli.py's _last_loads_by_group() reads state.path once per run and hands each group's slice of last_balance.load_vector (re-keyed to plain topology.Disk.key, or None if that group has no recorded balance yet) to evaluate_group_gates() as last_load – both show-load and plan do this identically (cli.py calls the one function, not two copies). evaluate_group_gates() itself needed no change at all: it already took last_load as a parameter and treated None correctly (see docs/internals/15-state.md for the read side, and this page's next section for what is still missing).

What this does not yet do. state.json's last_balance is only ever *read* here – nothing in this codebase calls state.with_recorded_balance() yet, because nothing executes a migration yet (execute.py, phase 7). Until then, last_balance.load_vector for any group stays exactly what it was the last time a human (or a test) wrote it by hand; a config with no state.json on disk, or a fresh install, still behaves exactly as this page originally described – last_load=None, drift gate skipped, decision reduces to "reserve override, else imbalance". The wiring is real; the writer that would make it self-sustaining is not built yet.

show-load's gate line is diagnostic, not a real decision

pve-storage-drs show-load prints each group's verdict (Group <name> → ACT: <reason> / → NO ACTION: <reason>) reusing GateDecision.reason verbatim — deliberately the single source of truth for the wording, rather than a second, similarly-but-not-identically phrased string living in cli.py.

Its verdict now reflects real drift history whenever `state.json` has one, exactly like `plan`'s does — but `show-load` still isn't what `apply` (not yet written) will actually gate an execution on; it is `plan`/`apply`'s own gate evaluation, run at the moment a plan is built or applied, that is authoritative.

Cooldowns are not this module's job

`gates.cooldown_per_disk`/`cooldown_per_storage` decide which *individual* disks or storages are movable once a plan is already being built — the same kind of decision (C2)'s other pin reasons make in `topology.py`, not a group-wide act/no-act verdict. They need `state.json`'s per-disk/ per-storage "last moved at" timestamps, which do not exist yet either, so implementing them here now would be speculative machinery with no caller and no real timestamp source to test against. They belong with whichever module builds the solver's movable-disk set (`heuristic.py`/`schedule.py`, phase 4) when that is written.

The heuristic solver

What does this page answer? How does `heuristic.py` turn a group's loads into a target assignment, why is the section 5.4 objective its own function rather than baked into the search, and what does this module deliberately not do yet? Describes `proxmox_storage_drs/heuristic.py`.

`evaluate_assignment()` is the objective, kept separate from the search

Section 5.5 requires it explicitly: "the heuristic must use the **same** feasibility and objective functions as the MILP path so the two backends are directly comparable." `evaluate_assignment()` takes a `Group` and an arbitrary candidate `Assignment` (`dict[Disk.key, storage id]`, not necessarily reachable from the current state by any sequence of moves) and returns an `ObjectiveBreakdown` — the section 5.4 objective's five terms kept separate, not collapsed into a single number, because `explain` (not yet written) needs the arithmetic shown, and because the tests cross-checking this against section 14's worked example need to assert each term individually, not just a total that could be right for the wrong reasons.

This is the same design choice `reserve.py` made first: a pure function of *data*, with no notion of "the current run" baked in. `heuristic.py` reuses `reserve.compute_reserve_status()` directly for (C4)/(C5), passing it a `storage_of` callback closed over the candidate assignment being evaluated — the extension point `reserve.py`'s own docstring already anticipated ("the solver will pass a candidate assignment through the identical shape") before this module existed to use it.

Repair is unconditional, not weight-driven

Section 13: "the reserve is never traded against balance." The heuristic makes this literal rather than merely well-weighted: `_repair()` runs *before* `_descend()` and fixes any (C5) violation by moving disks off the worst-violating storage, one iteration at a time, regardless of what the objective weights say — there is no `beta`/`gamma` value large enough to switch this off. `objective.reserve_violation_penalty` still appears in `ObjectiveBreakdown.reserve_penalty_term`, but only for *reporting* a residual that repair could not fix (physically impossible, not merely unattractive) — by the time `_descend()` runs, every reserve violation `_repair()` could resolve already has been, so in the common case this term is exactly zero and does not influence descend's choices at all. `test_reserve_violation_is_repaired_even_with_beta_high_enough_to_forbid_balance_moves` is the regression test for this: `beta=100` (so no purely-cosmetic balance move could ever be worth a migration) still sees the repair move happen.

A candidate is judged by the group-wide total shortfall, not the source storage's own. Moving a disk off a violating storage always reduces *that* storage's own shortfall — it has fewer bytes, and if anything a smaller largest-disk requirement — which makes "does the source's shortfall fall" a tempting but wrong test: it says yes even when the disk only relocates the problem to whichever storage receives it, or makes the group's total worse. An earlier version of `_repair` used exactly that test and would oscillate a disk back and forth between two storages neither of which can fully hold it, "successfully" reducing the current

worst offender's shortfall on every iteration while never making net progress. Requiring the *group* total to strictly decrease fixes this and is also what makes the loop's iteration bound (movable disks times storages, generously sized rather than tightly) actually a correct termination argument rather than a lucky one. `test_repair_does_not_oscillate_when_no_target_can_fully_absorb_the_violation` is the regression test: a 3 TiB disk that cannot fit, alone, on either of two 8 TiB/`reserve_factor: 2.0` storages is moved exactly once (repairing what can be repaired, from a 3 TiB group-wide shortfall down to 1 TiB), not shuffled back and forth forever.

Descend explores swaps, not only single moves

Section 5.5 is explicit that swaps are not an optional refinement: "when both storages are near their capacity limit, no single move is feasible, and only an exchange of two disks can improve the balance." `_descend()` evaluates every single-disk move *and* every pairwise swap of two movable disks each iteration, applying whichever most reduces the objective, and stopping when nothing does or after `heuristic_iterations`. Both are plain re-evaluations of `evaluate_assignment()` against a trial dictionary — no separate swap-feasibility logic exists, because a swapped assignment is just another candidate the same objective function scores. `test_descend_uses_a_swap_when_no_single_move_is_feasible` constructs exactly the capacity-locked scenario the plan describes (two storages each with headroom smaller than any single disk) and checks the only reachable improvement — a swap — is the one found.

`objective.spread_metric`: two different quantities, not one rescaled

Section 5.4/(C6) offers two imbalance forms, and `evaluate_assignment()` implements both: `"l1"` (default) is `alpha * sum(e_s)`, every storage's absolute deviation from `u*` summed; `"minmax"` is `alpha * max(u_s)` – the plan's own `t >= u_s` for all `s`, the *raw* utilization of the single hottest storage. These are not the same quantity with a different aggregation: `max(u_s)` and `max(e_s)` can disagree, because a storage sitting far *below* `u*` produces a large deviation `e_s` that minmax was never meant to notice (the manual's own words: "indifferent to a second nearly-as-bad storage" — nearly-as-bad on the *high* side; a cold storage is not what minmax was designed to react to at all). `ObjectiveBreakdown` carries both `spread_e` (deviations) and `utilization` (raw `u_s`) regardless of which metric is active, computed once in the same loop, so switching metrics never means computing something the other mode didn't already have on hand.

REVIEW.md's Q-01 found this config knob silently ignored by an earlier version of this module (always L1, `objective.spread_metric` never read) — the same class of gap P-01/P-02 found in `pve.py`/`topology.py`. `test_spread_metric_minmax_is_indifferent_to_a_second_nearly_as_bad_storage` is the regression test, built directly from the manual's own claim: two scenarios with an identical hottest storage but a materially different second-worst one score identically under minmax and differently under l1.

Proof this reproduces the plan, not just itself

`tests/unit/test_heuristic.py` doesn't stop at "the assignment matches the prose." At the default weights it asserts `ObjectiveBreakdown.total` equals `2.533333` (three-move plan) and, at `beta_move_count: 0.50`, `3.158333` (two-move plan) — the exact totals `REVIEW.md` Appendix A independently re-derived by hand from the plan's own numbers, not values this module invented and then asserted against itself. Getting both to five decimal places is strong evidence the objective's five terms, their units (TiB for size, average in-flight I/O for load), and the search that picks among them are all correct together, not merely internally consistent.

What this pass deliberately does not do

- **"Polish" (heuristic step 4)** — reuniting a fragmented VM when doing so does not worsen imbalance beyond `gates.imbalance_threshold`. Not implemented: the section 14 fixture's exact three-move and two-move solutions are both reachable by repair+descend alone (proven by the tests above), because the objective's own `kappa` term already makes descend prefer co-location whenever it

is not too costly. Polish would matter for an affinity fix that needs three or more disks to rotate simultaneously — outside single-move/pairwise-swap reach — which the one fixture that exists to validate this module does not exercise. A real gap, tracked here rather than silently absent.

- **(C2) format-compatibility eligibility** — `topology.Storage` does not yet carry the storage type/format information that rule needs (it is resolved internally in `topology.py`'s `_default_format` but never exposed on `Storage`), so every group storage is currently an eligible target for every movable disk. The section 14 fixture is homogeneous (three identically-typed storages) and does not exercise this gap either.
- **CP-SAT/CBC coefficient scaling (section 5.5)** — belongs to `optimize.py`, not yet written. `evaluate_assignment()`'s plain floating-point objective is what the MILP path will need to agree with, not a stand-in for its own separately-scaled integer objective. `heuristic.py` is wired into `cli.py`'s `plan` command, together with `schedule.py` (section 8's move ordering) — see `95-schedule.md` and `docs/manual/27-plan.md`. `explain`/`apply` remain stubs (`docs/manual/30-safety-and-status.md`).

Ordering the moves

What does this page answer? How does `schedule.py` turn a target assignment into an ordered, transient-feasible sequence of moves, and what happens when no order exists? Describes `proxmox_storage_drs/schedule.py`.

One loop, reusing `heuristic.py`'s objective for the ratio

`order_moves()` implements section 8.2's pseudocode directly: while there are pending moves, find the ones that are transient-feasible right now, schedule the best by imbalance-reduction-per-byte (moves that resolve a currently-violating storage win outright, regardless of that ratio), apply it, repeat. "Imbalance reduction" is computed by calling `heuristic.evaluate_assignment()` twice (before/after the candidate move) and diffing `.imbalance_term` — not a second formula, the same one `heuristic.py` already computes, so a plan's ordering agrees with whatever `objective.spread_metric` is configured exactly the way the assignment that produced it did.

`cost_m` is `z_d` — and that is not an approximation today

Section 8.2's ratio is "imbalance reduction / `cost_m`". `cost_m` here is simply the disk's size in bytes, not `payback.py`'s real duration-based cost (mirror time plus, when `saferemove` is on, wipe time — and wipe *throughput* is a per-storage config value, so two candidate moves off different sources can have genuinely different wipe costs even for the same `z_d`). `migration.bwlimit_bytes_per_sec` is one global config value, so every move's *mirror* duration alone is `z_d / bwlimit`, a constant divided into every disk's size equally — dividing an imbalance reduction by `z_d` and dividing it by `z_d / bwlimit` produce the **same ordering** (`bwlimit` is a positive constant common to every candidate). So `cost_m = z_d` reproduces the true mirror-only ordering exactly, but is an approximation once a group has moves whose `saferemove` wipe cost differs enough to change the ranking `payback.py`'s full cost would produce. Scoring candidates by `payback.compute_move_cost()` instead of `z_d` is a natural follow-up, not yet done: this module predates `payback.py` and has not been revisited to consume it (REVIEW.md R-04).

The transient invariant, called with a single-move set always

Section 8.1 defines a generalized invariant over a whole in-flight set `M` specifically so it can be "implement[ed] as the single feasibility predicate and call[ed] with `M = {m}` for the sequential case" — this module is that sequential case. `transient_invariant_ok()` checks one move against `state`, the assignment as of immediately before that move starts; every previously-scheduled move is assumed **fully complete** by then (mirror finished, and any `saferemove` wipe finished, source genuinely freed) before this one begins. That is the correct model for `execution.max_concurrent_migrations: 1` (the default, and the plan's own

"recommended configuration") — see the next section for why it is deliberately not generalized to more than one in-flight move yet.

The min_free_bytes floor is folded into the transient check the same way (C5) folds it into the steady-state one (max(f_b * max(Z_b, z_d), min_free_bytes)) — the plan's own section 8.1 formula does not mention the floor, but there is no reason a storage's absolute minimum free space should stop applying just because a migration happens to be in flight.

The heuristic can accept a residual violation; the scheduler cannot execute one

heuristic.py's repair step can leave a storage still violating (C5) if no reachable assignment can fully clear it — a genuine, sometimes unavoidable outcome the objective's reserve_penalty_term reports rather than hides (see 90-heuristic.md). When order_moves() is asked to schedule the move that leads to exactly that residual state, the transient check correctly says no: landing a disk on a target that still would not satisfy (C5) is not transient-feasible, full stop, and this module never treats "the heuristic already decided this was the best available" as a reason to relax that. The result is a deadlock report for that move (section 8.3 option 3, "report... never force a move that breaches the reserve") — an honest "this cannot be safely executed", not a false all-clear. See test_plan_reports_a_deadlock_when_even_the_best_target_still_violates in tests/unit/test_cli.py for the exact scenario reproduced end to end.

final_assignment is the schedule's own truth, not the heuristic's target

ScheduleResult.final_assignment is the assignment produced by actually applying order in sequence, starting from the current placement — every disk, not only the moved ones. When nothing deadlocks it is identical to the heuristic's target assignment, but when deadlocked is non-empty (fully or, more subtly, only partially) it is not: a caller reporting an "after" utilization/spread must evaluate final_assignment, not heuristic.HeuristicResult.assignment, or it reports numbers for moves that were never actually scheduled (REVIEW.md R-02 — cli.py's plan originally did exactly this, correct only when order_moves() scheduled every move a plan proposed).

What this pass deliberately does not do

- **Concurrent scheduling** (execution.max_concurrent_migrations > 1). payback.py now provides the move-duration estimates this would need, but reasoning correctly about overlapping in-flight windows — which pairs of moves can safely run together under the generalized section 8.1 invariant — is not implemented. This module schedules strictly sequentially regardless of the configured value; section 8.1's own generalized invariant is what a future version would evaluate against the real in-flight set instead of {m}.
- **Ordering priority 2** ("moves that free space a later move needs") and **staging** (section 8.3 option 1). Both are refinements over a plan that is already feasible and safe without them — priority 2 only affects how well a plan front-loads its value if interrupted, and staging only helps in a genuine pebble-motion deadlock. Omitting them can only make this module report a deadlock in a case a more sophisticated scheduler would have found a way through; it can never produce an order this module thinks is safe but is not.
- **Cooldowns**, for the same reason gates.py doesn't implement them yet: state.json (section 11.2), which would record the timestamps a real cooldown check needs, does not exist.
- **The section 7.3 saturation check** — belongs with payback.py (implemented; see 96-payback.md for why the check itself is deferred).

Wired into plan, not yet into apply

cli.py's plan command calls heuristic.run_heuristic() then schedule.order_moves() for every group the gate says to act on, and renders the result (see docs/manual/27-plan.md). Nothing calls

`schedule.py` from `apply` yet, because `apply` itself does not exist (`execute.py`, phase 7+) — `plan` only ever computes and prints.

Migration cost and the payback test

What does this page answer? How does `payback.py` turn a scheduled move into a cost, a plan into a benefit, and the two into an accept/reject verdict — and why does a reserve-fixing plan always pass? Describes `proxmox_storage_drs/payback.py`.

Cost and benefit are computed from data other modules already have

`compute_move_cost()` needs only a `schedule.ScheduledMove` and its source `topology.Storage` — no new fetch, no new state. `duration_mirror` is `z_d / migration.bwlimit_bytes_per_sec`; `duration_wipe` is `z_d / saferemove_throughput` when `migration.account_saferemove_wipe` and the source has `saferemove` on, else zero. `compute_benefit_load_seconds()` takes two `heuristic.ObjectiveBreakdown.imbalance_term` values — the pre-plan and post-plan `E` — and multiplies their difference by `migration.payback_horizon_seconds`; `heuristic.HeuristicResult` already carries both (`initial_breakdown`/`breakdown`), so `cli.py`'s `plan` handler passes them straight through.

`headroom_src/headroom_dst`: a plan formula this project cannot fill in

Section 7.1 writes `duration_mirror_d = z_d / min(bwlimit, headroom_src, headroom_dst)`. Neither `headroom_src` nor `headroom_dst` is defined anywhere else in `IMPLEMENTATION_PLAN.md`, and no config field or `topology.Storage` attribute represents a per-storage effective throughput ceiling distinct from the one global `migration.bwlimit_bytes_per_sec`. `compute_move_cost()` uses `z_d / bwlimit` only — exactly what the formula reduces to whenever neither storage's own throughput is the binding constraint, which is the case `bwlimit` exists to enforce in the first place. This is recorded as a known simplification of the plan's own underspecified formula, not silently worked around.

The reserve-override exemption

Section 7's payback test weighs a move's cost against the *balance* benefit it buys. That framing has an edge it does not name: a move resolving an active (C4)/(C5) violation is not optional the way a balance-driven move is, and its "benefit" under the section 7.2 formula can easily be zero or even structurally zero — moving the only loaded disk in a two-storage group between the two storages changes which one carries it without changing `E` at all, no matter how urgently the move is needed for capacity reasons. Rejecting that move on economic grounds would contradict section 13's own "the reserve is never traded against balance," which `gates.py` already treats as absolute for the drift/ imbalance gates. `evaluate_plan_payback()` applies the identical rule here: **a plan containing any move with `resolves_reserve_violation=True` always passes the aggregate ratio test**, regardless of the computed `ratio`. The hard per-move `max_single_move_duration` rule is not exempted this way — section 7.3 lists it as applying "regardless of the aggregate test" precisely because it is an operational limit (a mirror that takes that long has other costs an economic ratio does not capture), not an economic one.

`test_plan_json_output` and `test_plan_json_output_accepts_payback_when_saferemove_is_off` in `tests/unit/test_cli.py` exercise both halves of this end to end: the same reserve-driven, zero-benefit move is accepted when its duration is within the limit and rejected (correctly, via the hard rule, not the economic one) when a slow `saferemove` wipe pushes it over `max_single_move_duration`.

What this pass deliberately does not do

- **The 3-retry re-solve-with-doubled-`beta`/`gamma` loop** (section 7.3) on aggregate payback failure. A real UX refinement — it converges on the smaller subset of high-value moves rather than abandoning the run — but not a correctness requirement: reporting accept/reject plainly already satisfies "reject a plan whose cost outweighs its benefit." Implementing the loop means re-invoking

heuristic.py/schedule.py with adjusted weights from inside cli.py's own orchestration, which is where it belongs when it is built, not half-done inside this module now.

- **Automatically dropping an individually-rejected move** and re-solving without it. Reported instead (MoveCost.exceeds_max_duration, PaybackResult.rejected_moves) — the same "report, never force" choice schedule.py's deadlock reporting already makes for an unschedulable move.
- **The section 7.3 saturation-ceiling defer check** (L_during(s) <= saturation_ceiling * N_s). It needs the forecaster's upper bound over a horizon equal to a specific move's own duration, which forecast.py's Forecaster protocol does not expose (it answers for window.lookback only); and no group in this project's own dogfooding cluster has storages[].saturation_load set, which the plan itself says makes this check's absence "fully supported... loses only this one advisory check." max_single_move_duration (implemented) and the transient reserve invariant (schedule.py, already enforced before a move is ever scheduled) are the two bounds section 7.3 calls "always active"; this deferred one is explicitly the best-effort extra.

Wired into plan, not yet into apply

cli.py's _handle_plan() computes a PaybackResult for every group the gate acts on, right after scheduling its moves, and both render functions show it (docs/manual/27-plan.md). compute_wipe_duration_seconds() is also used by verify-storages's existing wipe-time warning — one implementation of the formula, not two (AGENTS.md section 5). Nothing calls this module from apply, because apply does not exist yet (execute.py, phase 7+).